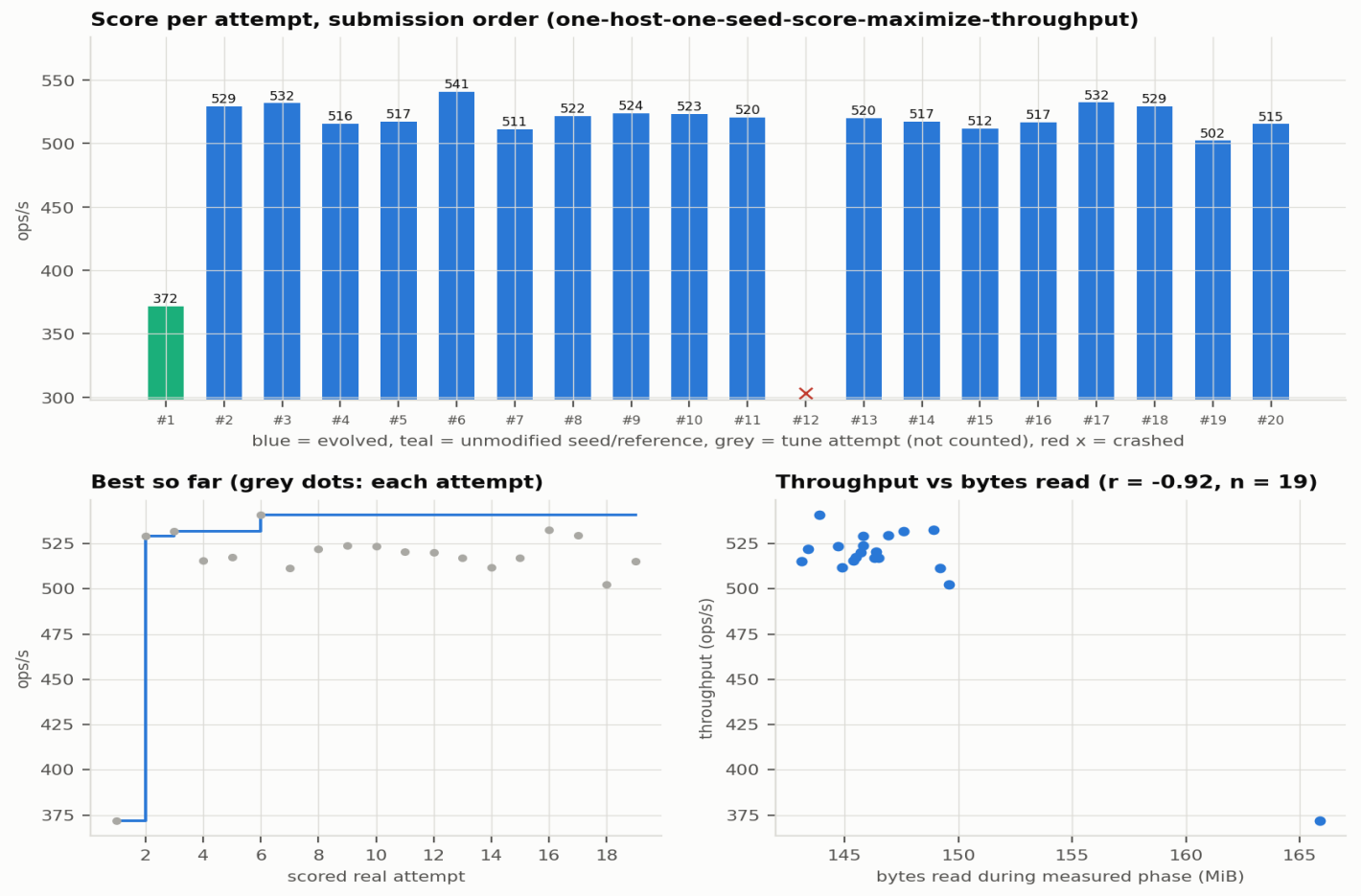
CORAL run 2026-10-20_173041 — one-host-one-seed-score-maximize-throughput

S3-FIFO is the only seed; score = absolute measured throughput (ops/s). Evaluated on CloudLab host pc320.emulab.net (2-core, ~2 GB Xeon; absolute throughput is not comparable with the earlier c220g1 runs or with the other hosts).

real attempts	20 (cap 20) | scored 19, crashed 1 | tune 0
stop reason	max_real_attempts
wall time (agent logs)	0.48 h
best attempt	#6 46ce09fd score 540.8 — agent-evolved
best evolved	#6 46ce09fd score 540.8
unmodified seed (s3fifo)	#1: 371.9
LLM usage	27.94 USD, 33k output tokens, 83 turns in 22 sessions; gateway requests 136 {400: 44, 200: 92}



All attempts (submission order)

#	commit	class	score	tput ops/s	read MiB	refaults	wall s	design / title
1	1bee2df2	real =s3fifo	371.9	371.9	166	18210	41	baseline: unchanged S3-FIFO seed (comment-only change)
2	3558bbf4	real	529.2	529.2	146	13202	41	S3-FIFO + scan-thread detection: tid adding >=25% of folios per 2048-add epoch -> folios go to scan_list evicted first (reserve cache/64); scanner acc
3	5397e6b9	real	531.8	531.8	148	13629	40	scan reserve cache/64 -> cache/256: evict scanner folios sooner to leave more room for GET working set
4	8d8b6320	real	515.5	515.5	145	13105	40	freq cap 3 -> 7 in main CLOCK: hot Zipfian GET pages survive more sweeps (on top of scan-thread detection, reserve cache/256)
5	af977adc	real	517.5	517.5	146	13127	41	don't add scan-list folios to ghost on eviction (GET re-read of scanner pages goes via small, not straight to main). freq cap back to 3, reserve cache
6	46ce09fd	real	540.8	540.8	144	12901	41	noise check: rerun of 5397e6b (scan detection, reserve cache/256, cap 3) unchanged except comment
7	4be0298e	real	511.3	511.3	149	14076	41	scanner detection threshold 25% -> 12.5% of epoch adds (more aggressive scan classification); base 5397e6b
8	bd00a397	real	521.8	521.8	143	12799	41	scanner threshold 3/8 of epoch adds (stricter; 1/8 hurt -> 511, suspect GET-thread misclassification); base 5397e6b
9	2f82d467	real	523.9	523.9	146	13152	41	small FIFO target cache/15 -> cache/10 (longer probation for GET pages before eviction); base 5397e6b
10	8fcffaa0	real	523.4	523.4	145	13004	40	GET-touched scan folios promoted to small (freq=1) instead of main: one GET hit must repeat before reaching main; base 5397e6b
11	e37aab7e	real	520.5	520.5	146	13333	41	main queue eviction: CLOCK iterate -> sampled lowest-freq (bpf_cache_ext_list_sample, sample_size=10, seed's unused evict_main); base 5397e6b
12	db8eabb2	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
13	8a85459e	real	520.0	520.0	146	13199	41	structural attempt 1/3 (fix compile: non-atomic u32 history bump) on refault-history: persistent ghost refault counter 4x cache, refaulted GET folios
14	3da618f3	real	517.2	517.2	146	13401	40	structural attempt 2/3 on refault-history: stricter admission - first refault (hist=2) goes to small, only hist>=3 goes to main with freq=min(hist-3,3
15	f702c59b	real	511.8	511.8	145	12947	40	structural attempt 3/3 on refault-history: attempt-1 admission (hist>=2 -> main, freq=min(hist-2,3)) with history map 16x cache instead of 4x (does lo
16	428ae155	real	516.9	516.9	146	13380	41	noise check #2: rerun of 5397e6b unchanged (prior 531.79, 540.79); all variants since land 511-524 - is the best real or lucky?
17	369d5fc0	real	532.3	532.3	149	13903	41	structural attempt 1/3 on scan-range reuse: scanner re-reading a ghost (recently evicted) page -> small queue probation instead of evict-first scan li
18	bc468c71	real	529.4	529.4	147	13450	40	structural attempt 2/3 on scan-range reuse: scanner ghost hits -> main (full CLOCK cycle) instead of small
19	c223475e	real	502.3	502.3	150	14062	41	small FIFO target cache/15 -> cache/40: GET readahead neighbours (~15KB/IO vs 4KB blocks) are never accessed; shorter small queue drops them sooner. b
20	25946fcf	real	515.3	515.3	143	12690	41	structural attempt 1/3 on unaccessed-first small eviction: pass A evicts only freq==0 small folios (GET readahead neighbours never touched), rotating

Crashed attempts (no score; real ones still count toward the cap)

#12 db8eabb2 (real): eval: compile failed: compile failed

Best evolved policy: #6 46ce09fd — score 540.8

Title: noise check: rerun of 5397e6b (scan detection, reserve cache/256, cap 3) unchanged except comment

Closest available seed by line similarity: s3fifo 89%. Lines that differ from s3fifo are highlighted below.

```
// rerun of 5397e6b for noise check
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX   (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    u8 list; /* LIST_SMALL / LIST_MAIN / LIST_SCAN */
};

#define LIST_SMALL 0
#define LIST_MAIN 1
#define LIST_SCAN 2

/*
 * Scan-thread detection: a thread that adds a large share of all folios in an
 * epoch of 2^EPOCH_SHIFT adds is treated as the scanner. Its folios go to a
 * dedicated scan list that is evicted first, and its accesses don't count as
 * hotness.
 */
#define EPOCH_SHIFT 11
#define SCAN_SHARE_THRESH ((1 << EPOCH_SHIFT) / 4)

struct tid_stat {
    u64 epoch;
    u64 cur;
    u64 prev;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u32);
    __type(value, struct tid_stat);
    __uint(max_entries, 8192);
} tid_stats SEC(".maps");

static u64 total_adds = 0;

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;
static u64 scan_list;
static s64 scan_list_size = 0;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

static inline bool tid_is_scanner(bool count_add)
{
    u32 tid = (u32)bpf_get_current_pid_tgid();
    u64 epoch;
    if (count_add)
        epoch = __sync_fetch_and_add(&total_adds, 1) >> EPOCH_SHIFT;
    else
        epoch = total_adds >> EPOCH_SHIFT;

    struct tid_stat *st = bpf_map_lookup_elem(&tid_stats, &tid);
    if (!st) {
        if (!count_add)
            return false;
        struct tid_stat ns = { .epoch = epoch, .cur = 1, .prev = 0 };
        bpf_map_update_elem(&tid_stats, &tid, &ns, BPF_NOEXIST);
        return false;
    }

    u64 cur = st->cur, prev = st->prev;
    if (st->epoch != epoch) {
        prev = (st->epoch + 1 == epoch) ? cur : 0;
        cur = 0;
        if (count_add) {
            st->prev = prev;
            st->cur = 0;
            st->epoch = epoch;
        }
    }
    if (count_add) {
        st->cur = cur + 1;
        cur++;
    }
    return prev >= SCAN_SHARE_THRESH || cur >= SCAN_SHARE_THRESH;
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
```

```
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0) {
        bpf_printk("cache_ext: init: Failed to create scan_list\n");
        return -1;
    }

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }

    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
    if (freq < 0)
        data->freq = 0;

    return freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->list = LIST_MAIN;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }
    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //  main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)   \
{                                       \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))        \
        return CACHE_EXT_CONTINUE_ITER;                 \
                                        \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))       \
        return CACHE_EXT_CONTINUE_ITER;                 \
                                        \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");        \
        return CACHE_EXT_CONTINUE_ITER;                 \
    }                                       \
                                        \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                \
        /*data->freq = 0;*/                             \
        return CACHE_EXT_EVICT_NODE;                    \
    }                                       \
                                        \
    return CACHE_EXT_CONTINUE_ITER;                     \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
```

```
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                            eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

static int bpf_score_scan_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data)
        return CACHE_EXT_CONTINUE_ITER;

    // Touched by a non-scanner thread: it's GET-hot, promote to main
    if (data->freq > 0) {
        data->list = LIST_MAIN;
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

static void evict_scan(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, scan_list, bpf_score_scan_fn, &opts,
                            eviction_ctx) < 0)
        return;

    if (__sync_fetch_and_sub(&scan_list_size, opts.nr_folios_continue) < 0)
        scan_list_size = 0;
    __sync_fetch_and_add(&main_list_size, opts.nr_folios_continue);
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //     main_list_size, small_list_size, cache_size);
    if (scan_list_size > (s64)(cache_size / 256)) {
        evict_scan(eviction_ctx, memcg);
        if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
            return;
    }

    if (small_list_size >= cache_size / 15 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main_iter(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data)
        return;

    // The scanner's own accesses are not a hotness signal
    if (tid_is_scanner(false))
        return;

    // Cap frequency at 3
    if (__sync_add_and_fetch(&data->freq, 1) > 3)
        data->freq = 3;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u8 ghost_val = 0;

    // if (bpf_cache_ext_list_del(folio)) {
    //  bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //  return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    // Don't return early, we want to delete the folio metadata regardless
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->list == LIST_MAIN)
        __sync_fetch_and_sub(&main_list_size, 1);
    else if (data->list == LIST_SCAN)
        __sync_fetch_and_sub(&scan_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);

    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //  bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    bool scanner = tid_is_scanner(true);
    if (scanner) {
        list_to_add = scan_list;
        new_meta.list = LIST_SCAN;
          sync fetch and add(&scan list_size, 1);
    } else if (folio_in_ghost(folio)) {
        list_to_add = main_list;
        new_meta.list = LIST_MAIN;
        __sync_fetch_and_add(&main_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.list = LIST_SMALL;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
```

```
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
        char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
            fprintf(stderr, "Invalid cgroup size\n");
            return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size)) {
```

```
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;
cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

Notes and caveats

- Hardware: pc320.emulab.net is a 2-core ~2 GB node; absolute numbers differ from other hosts and from the earlier c220g1 runs.
- Repeated evaluations of identical code vary by a few percent; differences of a few ops/s (or a few hundredths of tanh score) between attempts are within noise.
- No tune attempts were used.
- Best-by-score is a single measurement; re-measure before drawing conclusions about small gaps.